CSE371s (UG2023)
Control Engineering
Lab 1



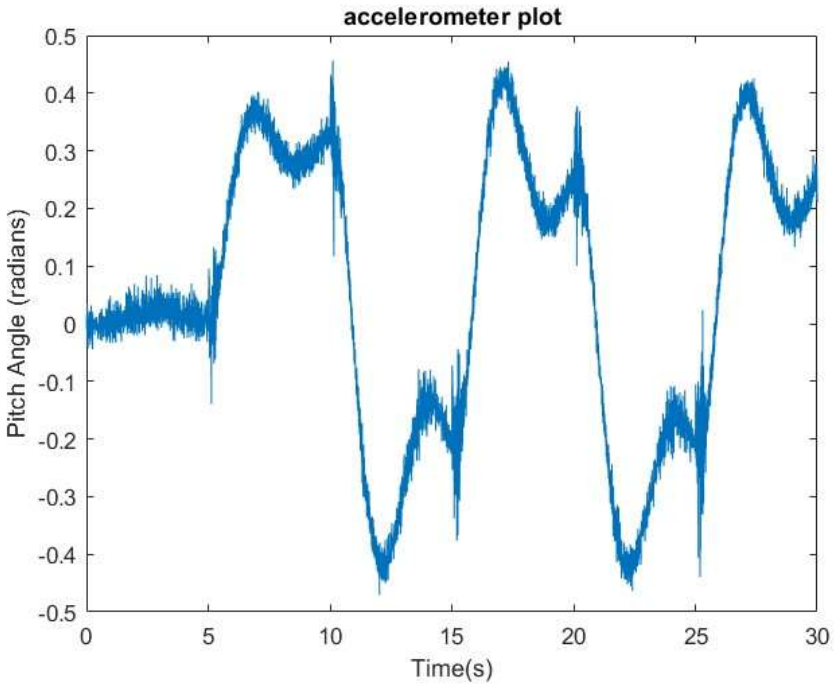


Abdelrahman sherif medhat
Section 1
2300100

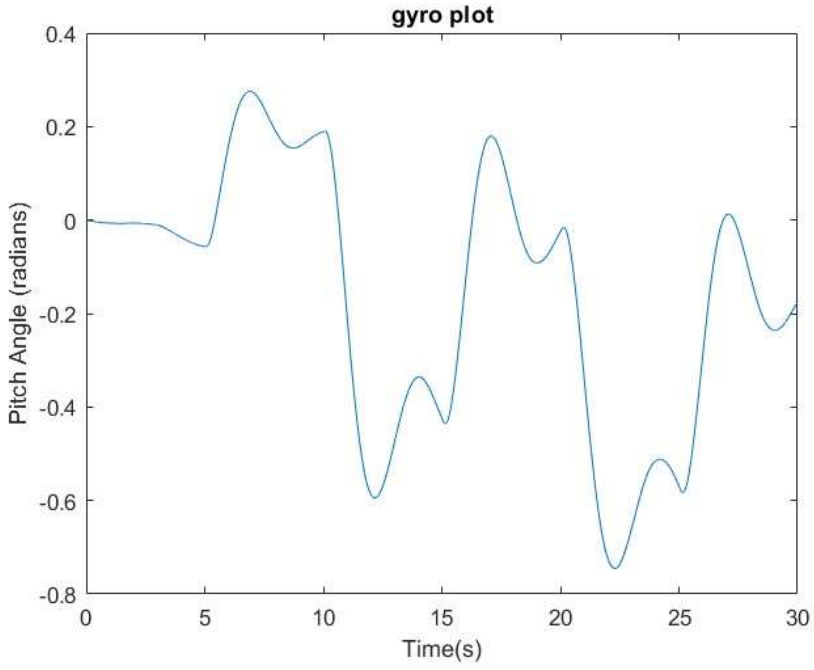
Part 1 Question (1):

plot pitch angle from accelerometer, gyroscope and encoder

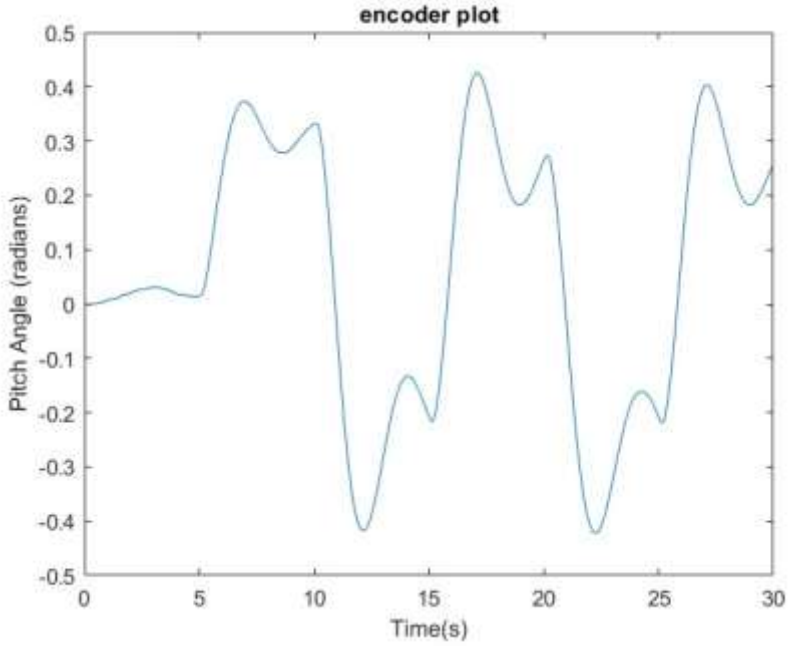
accelerometer:



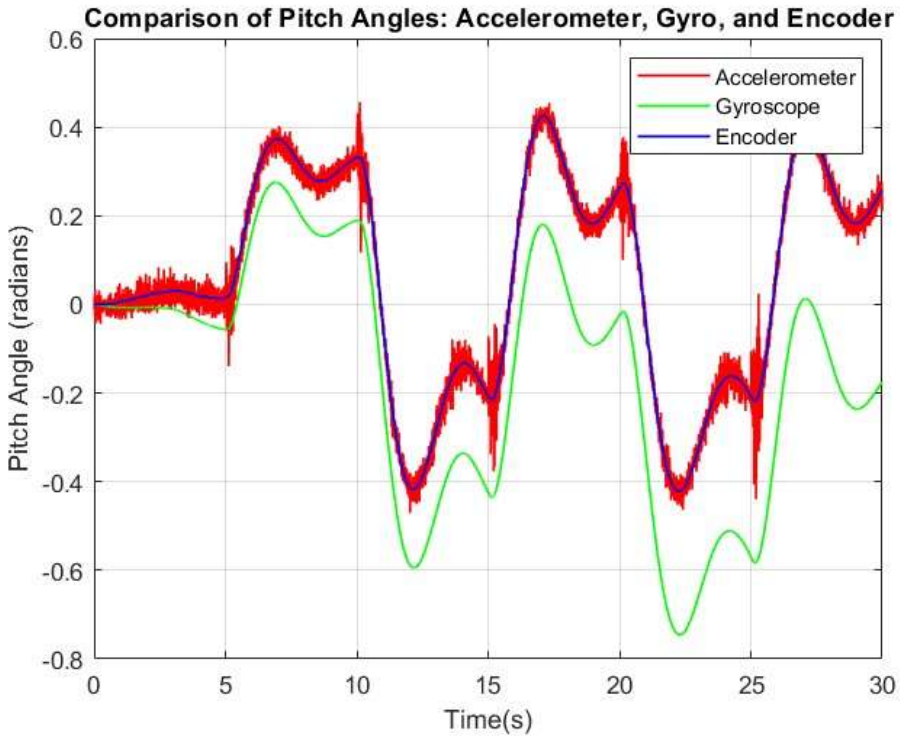
gyro:



Encoder:



Comparison:



Discuss the different characteristics of each measurement

1. Accelerometer

- Characteristic: High-Frequency Noise
- Analysis: The accelerometer calculates pitch angle based on the gravity vector. While it is stable over the long term (it doesn't drift away from the true value), it is extremely sensitive to vibration and rapid movements.
- Data Evidence:
    - Visual: The error plot shows a "fuzzy" or thick band of noise centered around zero. It fluctuates rapidly but maintains a constant average error close to zero.
- Pros: No drift, provides absolute angle relative to gravity.
- Cons: Very noisy signal; unusable for precise, smooth control without filtering.

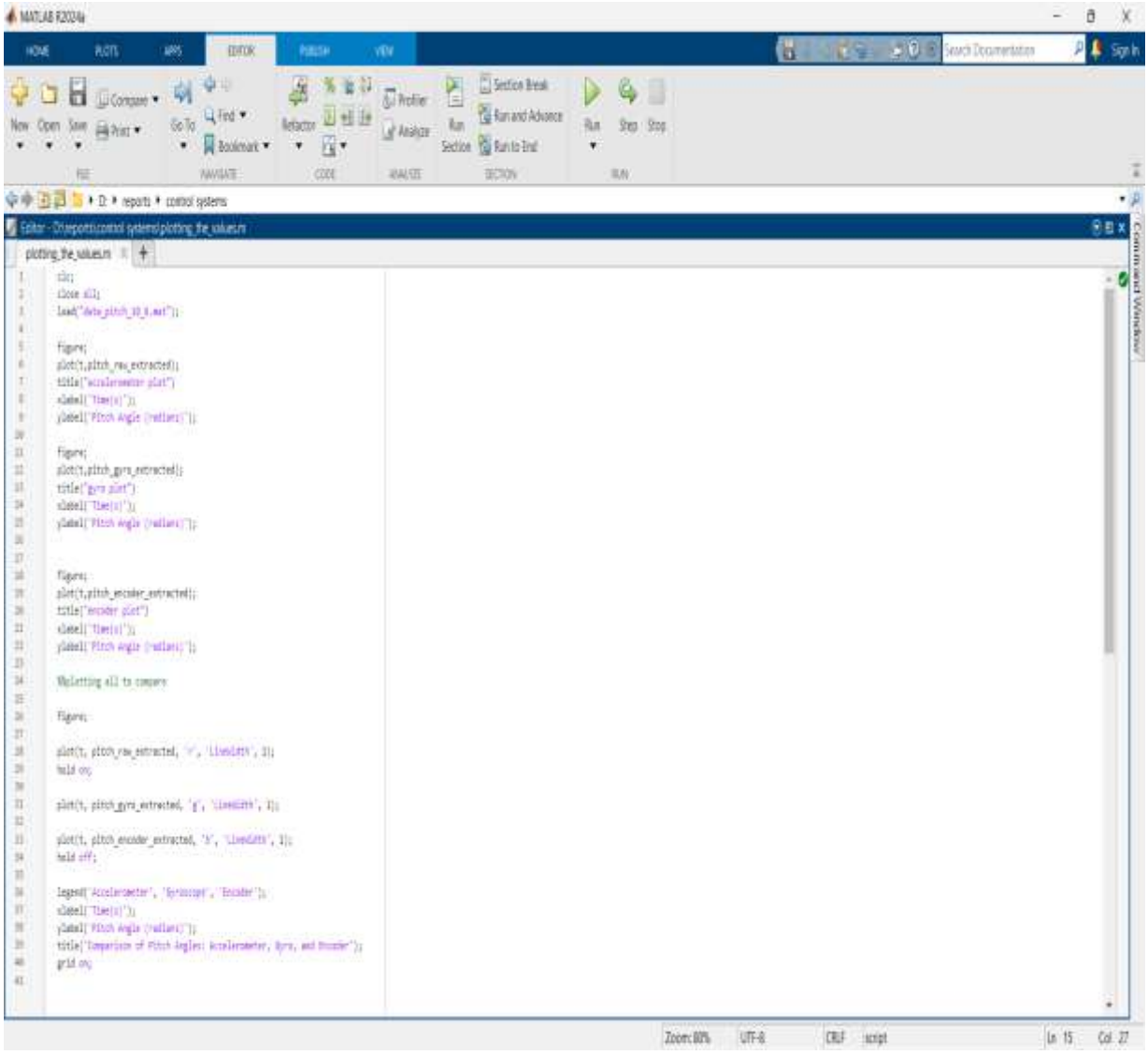
2. Gyroscope

- Characteristic: Low-Frequency Drift
- Analysis: The gyroscope measures angular velocity (rate of change), which is mathematically integrated to get the angle. This integration process accumulates small errors over time, causing the calculated angle to slowly "walk away" from the true value.
- Data Evidence:
- Visual: The error plot shows a smooth line that starts near zero but linearly decreases over time. Unlike the accelerometer, the line is smooth (low noise) but inaccurate in the long run.
- Pros: Very smooth signal, responsive to fast changes, low noise.
- Cons: Unbounded drift; the angle value becomes meaningless after a long period without correction.

3. Encoder (Blue/Reference)

- Characteristic: Ground Truth / High Accuracy
- Analysis: The encoder physically measures the rotation of the shaft. It is considered the "Ground Truth" for this dataset.
- Pros: Highly accurate, low noise, no drift.
- Cons: Often requires physical linkage to a stationary reference frame, which isn't always possible in mobile vehicles (like airplanes or drones), hence why we rely on Accelerometers and Gyros.

Plotting code



```
clc;
close all;
load("data_pitch_10_6.mat");

figure;
plot(t,pitch_raw_extracted);
title("accelerometer plot")
xlabel("Time(s)");
ylabel("Pitch Angle (radians)");

figure;
plot(t,pitch_gyro_extracted);
title("gyro plot")
xlabel("Time(s)");
ylabel("Pitch Angle (radians)");


figure;
plot(t,pitch_encoder_extracted);
title("encoder plot")
xlabel("Time(s)");
ylabel("Pitch Angle (radians)");

%plotting all to compare

figure;

plot(t, pitch_raw_extracted, 'r', 'LineWidth', 2);
hold on;

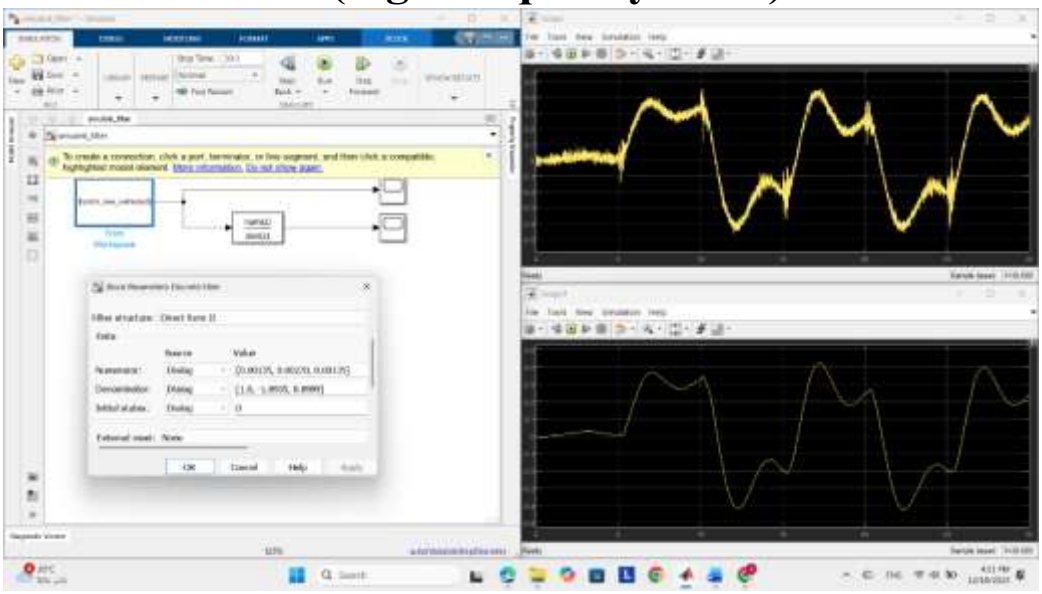
plot(t, pitch_gyro_extracted, 'g', 'LineWidth', 2);

plot(t, pitch_encoder_extracted, 'b', 'LineWidth', 2);
hold off;

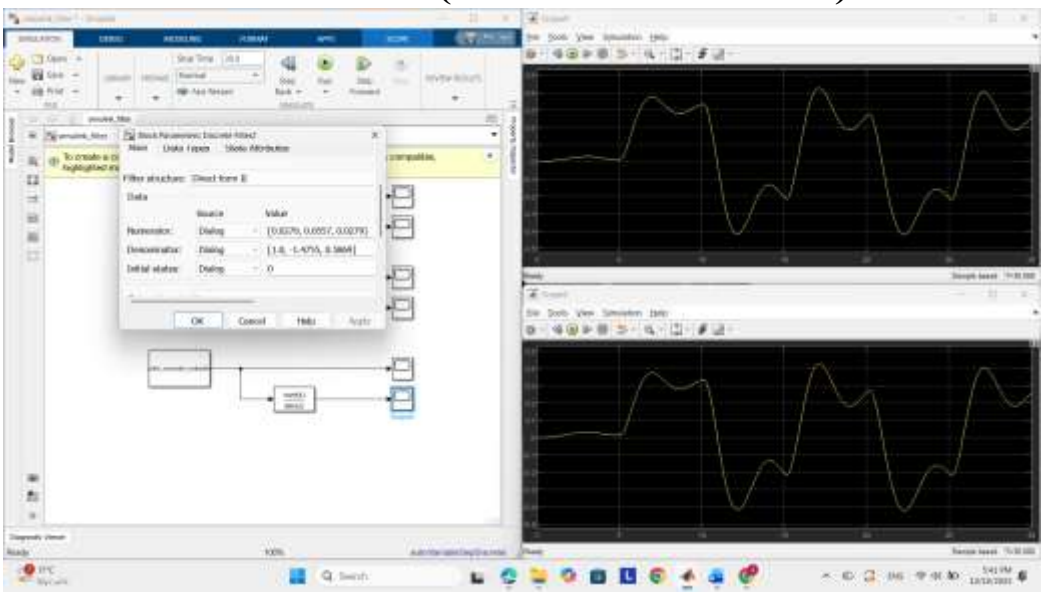
legend('Accelerometer', 'Gyroscope', 'Encoder');
xlabel('Time(s)');
ylabel('Pitch Angle (radians)');
title('Comparison of Pitch Angles: Accelerometer, Gyro, and Encoder');
grid on;
```

Filter design

Accelerometer: (high frequency noise)



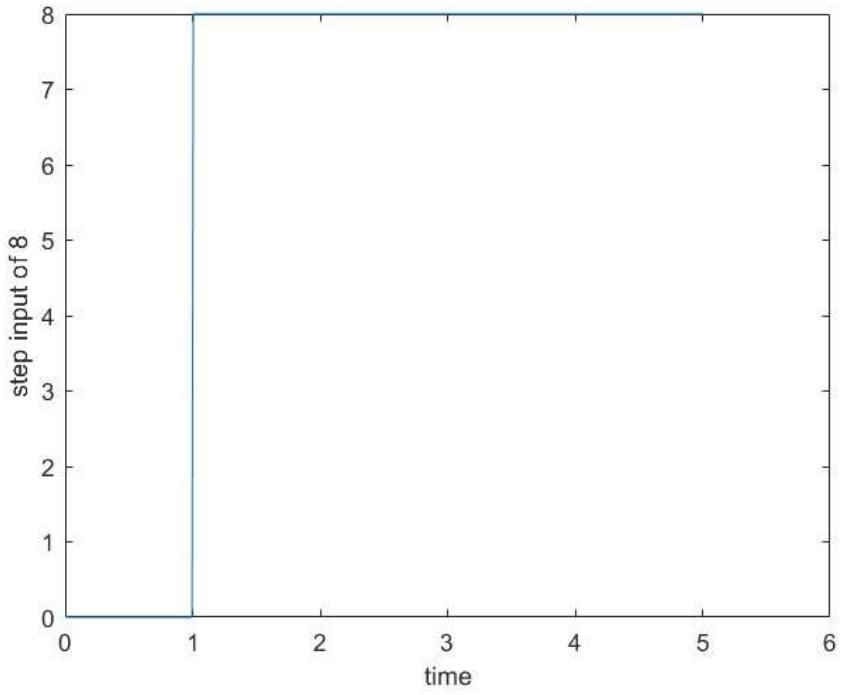
encoder: minor noise(doesn’t need much)

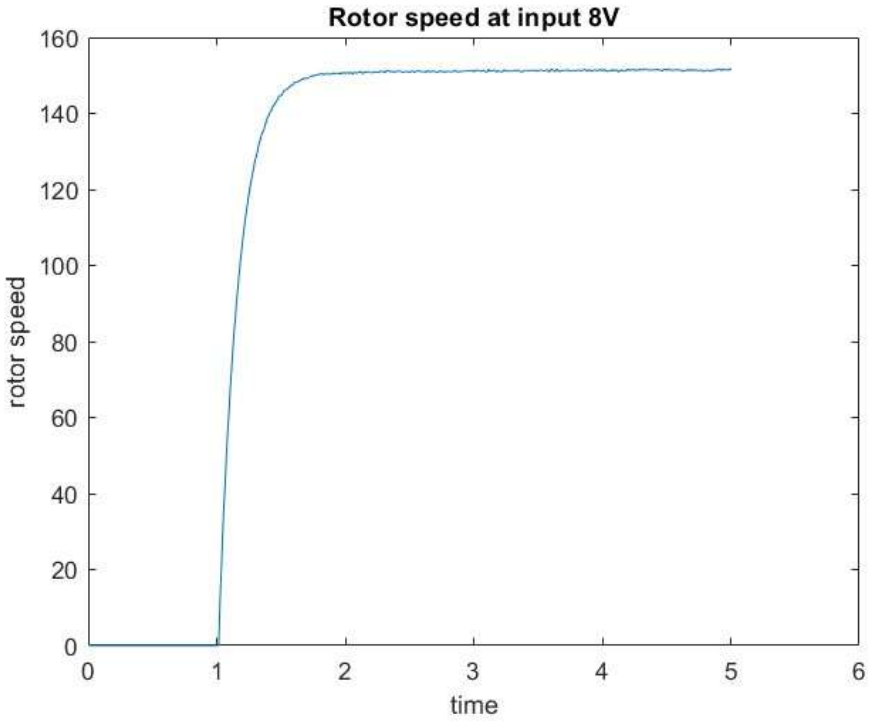


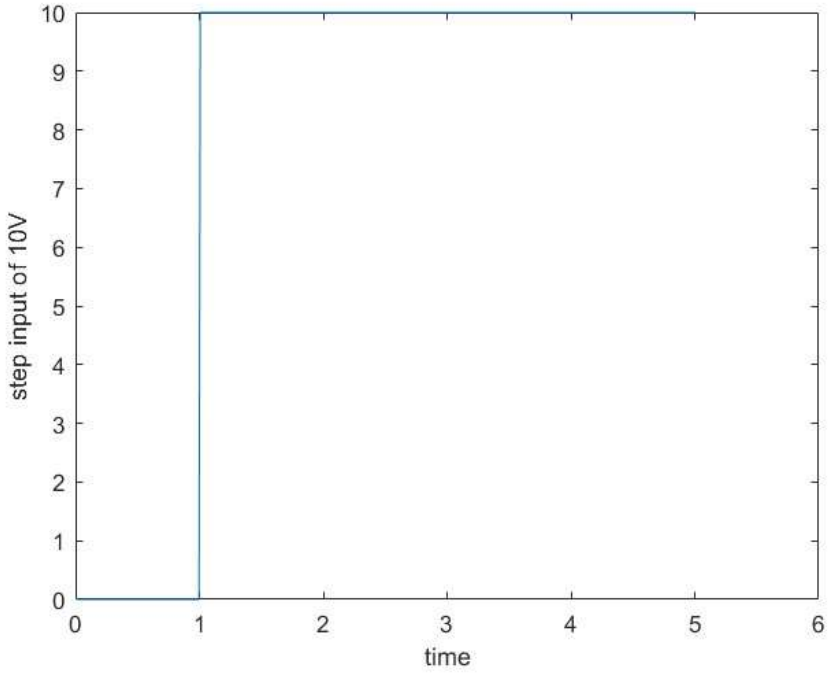
Gyro: (doesn’t need filter but needs correction)

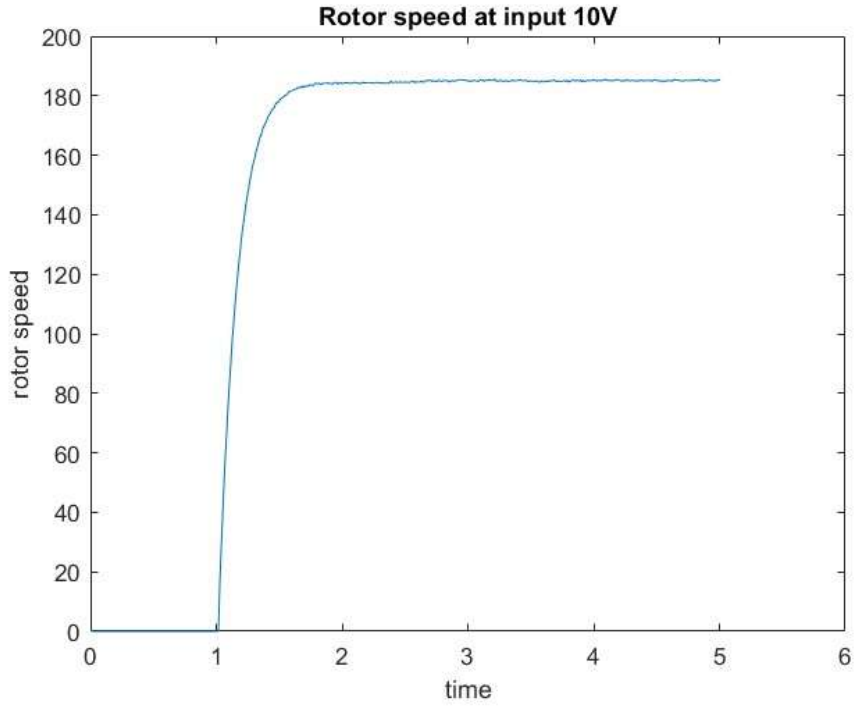
Part 1 Question (1):

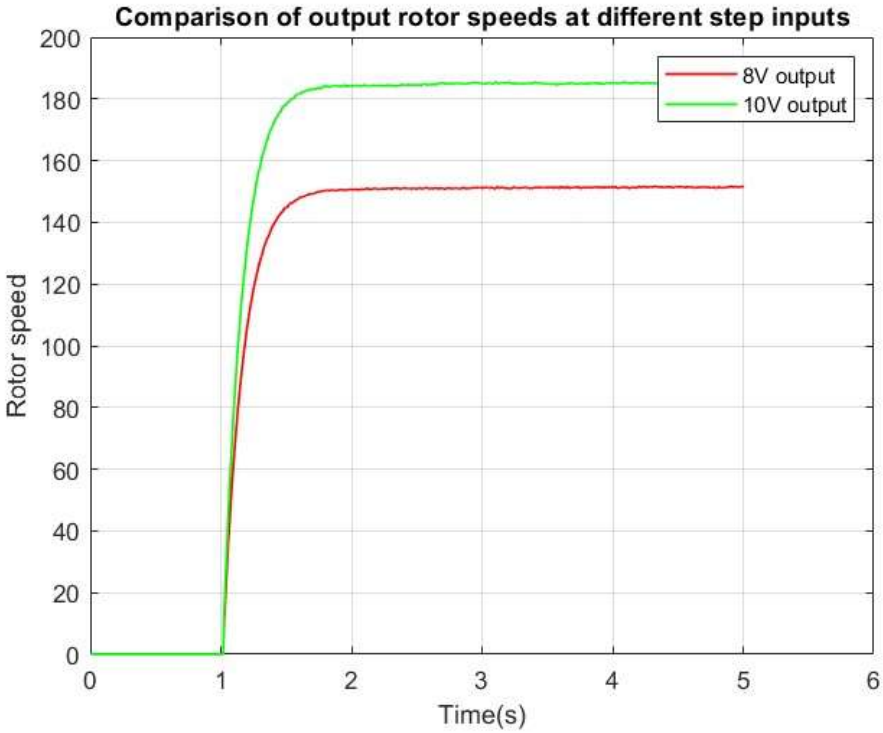
Plot the step response between motor voltage and rotor speed

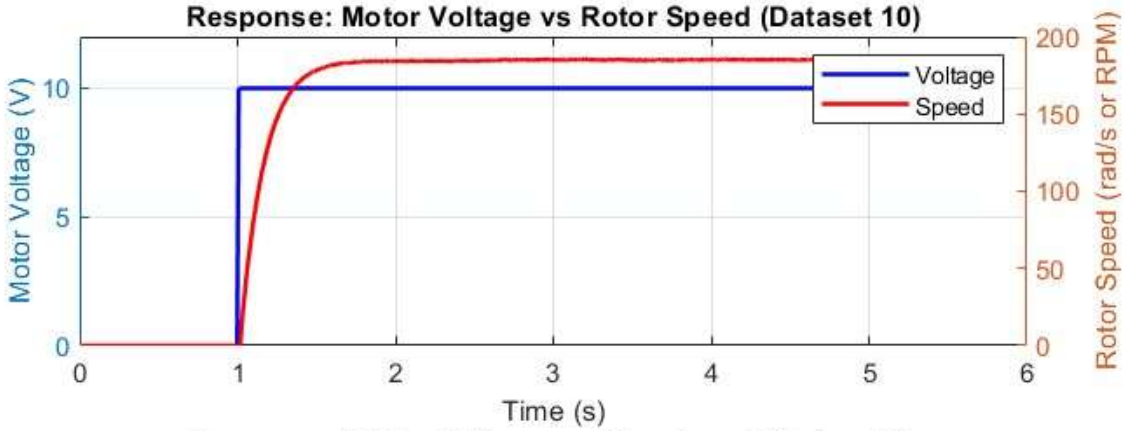


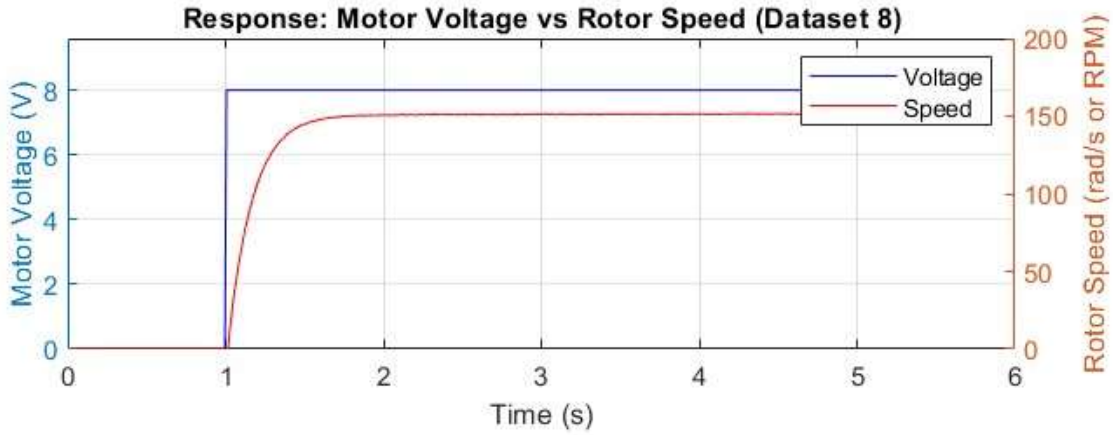



step input of 10V
time
0
1
2
3
4
5
6
7
8
9
10


Rotor speed at input 10V
rotor speed
time
0
20
40
60
80
100
120
140
160
180
200
1
2
3
4
5
6


Comparison of output rotor speeds at different step inputs
8V output
10V output
Rotor speed
Time(s)
0
20
40
60
80
100
120
140
160
180
200
0
1
2
3
4
5
6


Response: Motor Voltage vs Rotor Speed (Dataset 10)
Voltage
Speed
Motor Voltage (V)
Rotor Speed (rad/s or RPM)
Time (s)
0
5
10
0
50
100
150
200
0
1
2
3
4
5
6


Response: Motor Voltage vs Rotor Speed (Dataset 8)
Voltage
Speed
Motor Voltage (V)
Rotor Speed (rad/s or RPM)
Time (s)
0
2
4
6
8
0
50
100
150
200
0
1
2
3
4
5
6

If the motor is a first order system identify its parameters

$G(s) = \frac{K}{\tau s+1}$ $\Rightarrow$ $R(s) = \frac{1}{s}$

$C(s) = \frac{K/\tau}{s(\frac{1}{\tau}s)} = \frac{A}{s} + \frac{B}{\frac{1}{\tau}+s}$

$K = \frac{\Delta y}{\Delta u} = \frac{185.14}{10} = 18.514$

$A(\frac{1}{\tau}+s) + B(s) = \frac{K}{\tau}$

$C(s) = \frac{K}{s} + \frac{-K}{\frac{1}{\tau}+s}$

$A = K$ $\quad s=0$

$-\frac{1}{\tau}B = \frac{K}{\tau}$ $\quad s = -1/\tau$

$B = -K$

$c(t) = K - Ke^{-t/\tau} = K(1-e^{-t/\tau})$

at $c(\infty) = K =$

$\tau \rightarrow$ time to reach 63.2% $\Rightarrow$ $185.14 \times 0.632 = 116.92$ rad/s

Start of step input at $t = 1$ sec.

$y = 116.92$ at $t = 1.17$ s

$\tau = 0.17$ sec | $K = 18.51$

Develop matlab code to take input /output step response data and return the estimated parameters (K , tau)

system_values_calculation.m

```
clear;
close all;


load('data_pitch_10_8.mat');

function [K, tau] = estimate_first_order_params(time, input_signal, output_signal)

    d_input = diff(input_signal);
    threshold = max(abs(d_input)) * 0.5; % Threshold to detect the step
    step_idx = find(abs(d_input) > threshold, 1, 'first');

    if isempty(step_idx)
        error('No step input detected.');
    end

    idx_start = step_idx + 1;
    t_start = time(idx_start);

    u_initial = mean(input_signal(1:step_idx)); % Average before step
    u_final = mean(input_signal(end-50:end));   % Average at steady state
    delta_u = u_final - u_initial;

    % Output Step Size
    y_initial = output_signal(idx_start);       % Value at moment of step
    y_final = mean(output_signal(end-50:end));  % Steady state value
    delta_y = y_final - y_initial;

    % 3. Calculate Gain (K)
    K = delta_y / delta_u;

    % 4. Calculate Time Constant (tau)
    % Target value is 63.2% of the way from initial to final
    y_target = y_initial + 0.632 * delta_y;

    % Find the index where output crosses the target value
    % We restrict search to times AFTER the step occurred
    post_step_range = idx_start:length(output_signal);
    [~, min_idx] = min(abs(output_signal(post_step_range) - y_target));

    % Map back to global index
    idx_63 = post_step_range(min_idx);
    t_63 = time(idx_63);

    tau = t_63 - t_start;
```

Editor - D:\reports\control systems\system_values_calculation.m

system_values_calculation.m

```
    % Display Results
    fprintf('-------------------------------------------------\n');
    fprintf('First-Order System Identification Results:\n');
    fprintf('Step Time:    %.4f s\n', t_start);
    fprintf('Input Step:   %.2f -> %.2f (Delta: %.2f)\n', u_initial, u_final, delta_u);
    fprintf('Steady State: %.2f\n', y_final);
    fprintf('Gain (K):     %.4f\n', K);
    fprintf('Time Const (tau): %.4f s\n', tau);
end



%%10V dataset

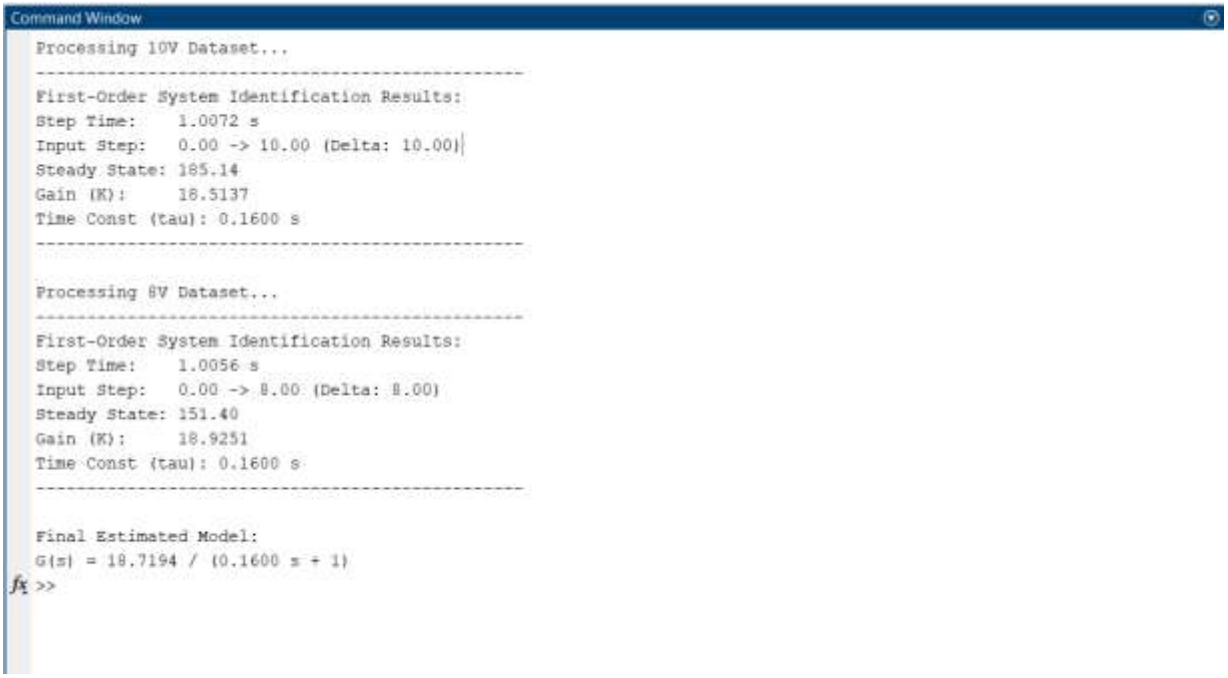

fprintf('Processing 10V Dataset...\n');
[K10, tau10] = estimate_first_order_params(time_10, input_10, output_10);

%% 8V dataset
fprintf('\nProcessing 8V Dataset...\n');
[K8, tau8] = estimate_first_order_params(time_8, input_8, output_8);

%% average
K_avg = (K10 + K8) / 2;
tau_avg = (tau10 + tau8) / 2;

fprintf('\nFinal Estimated Model:\n');
fprintf('G(s) = %.4f / (%.4f s + 1)\n', K_avg, tau_avg);
```

Output


Command Window
Processing 10V Dataset...
First-Order System Identification Results:
Step Time: 1.0072 s
Input Step: 0.00 -> 10.00 (Delta: 10.00)
Steady State: 185.14
Gain (K): 18.5137
Time Const (tau): 0.1600 s
Processing 8V Dataset...
First-Order System Identification Results:
Step Time: 1.0056 s
Input Step: 0.00 -> 8.00 (Delta: 8.00)
Steady State: 151.40
Gain (K): 18.9251
Time Const (tau): 0.1600 s
Final Estimated Model:
G(s) = 18.7194 / (0.1600 s + 1)
>>